

DOCUMENTAÇÃO TÉCNICA

Projeto MLP Backpropagation

Classificação do Dry Bean Dataset

Disciplina: Inteligência Computacional

Autores: Yago e Davi | 2026

Versão 1.0 | Arquivo: `mlp_dry_bean.py`

Esta documentação descreve a arquitetura, o fluxo de execução, os módulos de código, as escolhas de design e os resultados do projeto de classificação de grãos de feijão utilizando Redes Neurais MLP com Backpropagation, implementado em Python com scikit-learn.

1. Visão Geral do Projeto

1.1 Objetivo

Desenvolver, treinar e avaliar Redes Neurais MLP (Perceptron Multicamadas) para o problema de classificação multiclasse do **Dry Bean Dataset**, investigando sistematicamente os fatores que influenciam o desempenho: inicialização de pesos, hiperparâmetros, topologia, dados e atributos.

1.2 Escopo do Projeto

Aspecto	Detalhe
Problema	Classificação multiclasse supervisionada (7 classes)
Dataset	Dry Bean Dataset – UCI ML Repository (ID 602)
Linguagem	Python 3.12+
Framework ML	scikit-learn (MLPClassifier, Pipeline, StratifiedKFold, ...)
Visualização	matplotlib 3.x + seaborn
Arquivo Principal	mlp_dry_bean.py
Saída	Tabelas CSV + gráficos PNG + relatório TXT em resultados_mlp/

1.3 Requisitos do Sistema

Para executar o projeto, as seguintes dependências devem ser instaladas (via pip ou uv):

```
uv add numpy pandas scikit-learn matplotlib seaborn ucimlrepo openpyxl
```

Versões mínimas recomendadas: Python 3.10+, scikit-learn 1.3+, matplotlib 3.7+, numpy 1.24+.

2. Estrutura do Código

O projeto está organizado em um único arquivo Python (mlp_dry_bean.py), estruturado em funções independentes por questão, facilitando a manutenção e a reprodutibilidade dos experimentos.

2.1 Organização das Funções

Função	Responsabilidade	Questão
<code>carregar_dados()</code>	Carrega o Dry Bean Dataset do UCI (ucimlrepo ou fallback <code>OpenML</code>)	Q0
<code>salvar_fig()</code>	Salva figuras matplotlib no diretório de saída	Utilitário
<code>tabela_resultados()</code>	Exporta DataFrame para CSV	Utilitário
<code>treinar_e_monitorar()</code>	Treina MLP época a época coletando histórico	Utilitário
<code>questao_1()</code>	Exploração inicial: 5 seeds diferentes	Q1
<code>questao_2()</code>	Grid Search: LR × Momentum com SGD	Q2
<code>questao_3()</code>	Variação de topologia: camadas × neurônios	Q3
<code>questao_4()</code>	Influência da quantidade de dados de treino	Q4
<code>questao_5()</code>	Seleção e importância de atributos	Q5
<code>questao_6()</code>	Validação inicial no conjunto de teste	Q6
<code>questao_7()</code>	Validação cruzada k=10 folds estratificados	Q7
<code>gerar_resumo()</code>	Gera relatório resumo em TXT	Relatório
<code>main()</code>	Orquestra toda a execução sequencial	Entrada

2.2 Constantes Globais

Constante	Valor	Descrição
<code>BASE_DIR</code>	<code>Path.cwd()</code>	Diretório base de execução
<code>OUT_DIR</code>	<code>resultados_mlp/</code>	Diretório de saída dos artefatos gerados
<code>PALETTE</code>	<code>7 cores HEX</code>	Paleta de cores para os gráficos
<code>SEEDS</code>	<code>[0, 42, 7, 123, 999]</code>	5 seeds para Q1 (inicialização de pesos)
<code>CLASSES</code>	<code>Lista de strings</code>	Nomes das 7 variedades de feijão

3. Fluxo de Execução

3.1 Pipeline Completo

1. Carregamento	<code>carregar_dados()</code>	Download do Dry Bean Dataset via ucimlrepo (ID=602). Fallback: <code>fetch_openml</code> . <code>LabelEncoder</code> transforma os rótulos textuais em inteiros 0–6.
2. Pré-processamento	<code>StandardScaler().fit_transform()</code>	Normalização Z-score global (<code>media=0</code> , <code>std=1</code>) aplicada a todo o dataset X.
3. Divisão principal	<code>train_test_split(..., stratify=y)</code>	80% para experimentos (X_main) 20% para teste final fixo (X_test). Estratificação garante proporcionalidade entre as 7 classes.
4. Subdivisão treino/val	<code>train_test_split(..., test_size=0.25)</code>	Do conjunto X_main: 75% treino (Q1–Q2) 25% validação — resultando em 60/20/20 total.
5–11. Questões Q1–Q7	<code>questao_1() ... questao_7()</code>	Execução sequencial das 7 questões, cada uma gerando tabelas CSV e gráficos PNG.
12. Relatório	<code>gerar_resumo()</code>	Consolida os principais parâmetros e achados em <code>relatorio_resumo.txt</code> .

3.2 Executando o Projeto

Instalação com UV (Recomendado):

```
pip install uv && uv sync
uv run python mlp_dry_bean.py
```

Instalação com pip:

```
pip install numpy pandas scikit-learn matplotlib seaborn ucimlrepo openpyxl
python mlp_dry_bean.py
```

4. Detalhes de Implementação por Questão

Q1 – questao_1(X_train, y_train)

Parâmetro	Valor
hidden_layer_sizes	(128, 64)
activation	'relu'
solver	'adam'
learning_rate_init	0.001
max_iter	100
random_state	Variável (SEEDS = [0, 42, 7, 123, 999])

Saídas geradas: q1_resultados.csv, q1_curvas_convergencia.png

Lógica: Para cada seed, instancia um novo MLPClassifier, treina com fit(), e registra a curva de loss (loss_curve_), acurácia e F1-macro no conjunto de treino.

Q2 – questao_2(X_train, y_train, topology)

Implementa Grid Search manual com **solver='sgd'** e warm_start=True para treino época a época. A cada época, verifica se $\text{loss} \leq 0.001$; em caso positivo, registra o número de épocas até convergência.

Saídas geradas: q2_resultados.csv, q2_heatmap_grid.png, q2_curvas_convergencia.png

Q3 – questao_3(X_train, y_train, X_val, y_val, lr, mom)

Testa todas as combinações de **N camadas** $\in \{1, 2, 3\} \times$ **N neurônios** $\in \{10, 30, 50, 80, 100\}$. Para cada topologia, registra loss de treino e validação por época, acurácia e F1-Macro no conjunto de validação. As 4 melhores são selecionadas por F1-Macro.

Saídas: q3_resultados.csv, q3_top4.csv, q3_heatmap_f1.png, q3_top4_curvas.png

Q4 – questao_4(X, y, best_topo, lr, mom)

Varia o tamanho do conjunto de treinamento em **5 frações**: 20%, 40%, 60%, 80%, 100%. A cada fração, um subconjunto estratificado é extraído de X_main (80% do total). O conjunto de teste (20% fixo) permanece constante para comparação.

Saídas: q4_resultados.csv, q4_curvas_generalizacao.png

Q5 – questao_5(X_main, y_main, X_test, y_test, feature_names, ...)

Quatro estratégias de seleção de atributos: Todos (16), Top 12 permutação, Top 8 permutação, Sem alta correlação, Aleatório (5).

Saídas: q5_resultados.csv, q5_atributos.png

Q6 – questao_6(X_main, y_main, X_test, y_test, top4_df, ...)

Avalia as 4 melhores topologias da Q3 no conjunto de teste real. Para cada topologia, registra loss e acurácia por época (treino + validação) e métricas finais no teste. Gera a matriz de confusão da melhor configuração.

Saídas: q6_resultados.csv, q6_curvas_validacao.png, q6_confusion_matrix.png

Q7 – questao_7(X, y, best_topo, lr, mom, best_cols)

Validação cruzada com **StratifiedKFold(n_splits=10)**. A cada fold, aplica um StandardScaler independente (fit no treino, transform no teste) para evitar data leakage. Registra loss, acurácia e F1-Macro para cada partição.

Saídas: q7_resultados_folds.csv, q7_estatisticas.csv, q7_validacao_cruzada.png

5. Artefatos Gerados

5.1 Arquivos CSV (Tabelas de Resultados)

Arquivo	Conteúdo	Q
q1_resultados.csv	5 treinamentos: loss, acurácia, F1, épocas	1
q2_resultados.csv	Grid LR×Momentum: épocas, acurácia, convergência	2
q3_resultados.csv	Todas as topologias testadas	3
q3_top4.csv	As 4 melhores topologias selecionadas	3
q4_resultados.csv	Métricas por fração de dados	4
q5_resultados.csv	Métricas por subconjunto de atributos	5
q6_resultados.csv	Comparação das 4 redes no conjunto de teste	6
q7_resultados_folds.csv	Métricas de cada fold da validação cruzada	7
q7_estatisticas.csv	Média e desvio padrão das métricas do k-fold	7
relatorio_resumo.txt	Resumo textual de todos os experimentos	Geral

5.2 Gráficos PNG

Arquivo	Conteúdo	Q
q1_curvas_convergencia.png	Loss por época e acurácia final (5 seeds)	1
q2_heatmap_grid.png	Heatmaps de épocas e acurácia no Grid Search	2
q2_curvas_convergencia.png	Curvas de perda por LR, agrupadas por Momentum	2
q3_heatmap_f1.png	Heatmap de F1-Macro por topologia	3
q3_top4_curvas.png	Curvas treino/val das 4 melhores topologias	3
q4_curvas_generalizacao.png	F1, Acurácia e Loss vs. fração de dados	4
q5_atributos.png	Importância por permutação e F1 por subconjunto	5
q6_curvas_validacao.png	Loss/Acurácia treino×teste por topologia	6
q6_confusion_matrix.png	Matriz de confusão da melhor topologia	6
q7_validacao_cruzada.png	Curvas de fold, barras F1 e boxplot de métricas	7

6. Boas Práticas e Observações de Implementação

◆ *Data Leakage:*

O `StandardScaler` é sempre ajustado (fit) apenas no conjunto de treino e aplicado (transform) nos demais. Isso previne vazamento de informação da validação/teste para o treinamento — prática crítica para estimativas realistas de desempenho.

◆ *Estratificação:*

Todas as divisões (`train_test_split`, `StratifiedKFold`) usam `stratify=y`, garantindo que a proporção das 7 classes seja mantida em todas as partições.

◆ *Reprodutibilidade:*

`random_state=42` é usado como padrão em todos os experimentos (exceto Q1 onde as seeds variam intencionalmente). Isso garante resultados reprodutíveis.

◆ *warm_start=True:*

Usado nas questões Q2–Q7 para permitir treinamento época a época com coleta de métricas intermediárias, simulando um loop de treinamento personalizado.

◆ *Tratamento de exceções:*

Blocos `try/except` envolvem cada passo de treinamento para lidar graciosamente com casos de divergência numérica (ex.: `LR=0.5` com algumas configurações).

◆ *Escala logarítmica nos gráficos:*

As curvas de loss são plotadas em escala logarítmica (`yscale='log'`) para melhor visualização da convergência, especialmente quando valores iniciais e finais diferem muito.

6.1 Limitações Conhecidas

- O `MLPClassifier` do `scikit-learn` não expõe controle granular da Softmax — ela é calculada internamente pela função de perda `log_loss`.
- A importância por permutação (Q5) é computacionalmente intensiva; para datasets maiores, considere SHAP ou LIME.
- O `warm_start` com `max_iter=1` pode produzir avisos de convergência; esses são suprimidos com `warnings.filterwarnings('ignore')`.
- Os resultados de Q2–Q7 utilizam SGD em vez de Adam, para permitir controle explícito do momentum.

7. Configuração Final Recomendada

Parâmetro	Valor Final	Fonte
Topologia	(30,)	Q7 – melhor na validação cruzada
Ativação ocultas	ReLU	Q1 – estabilidade comprovada
Otimizador	SGD	Q2 – compatível com momentum explícito

Taxa de Aprendizado	0.1	Q2 – melhor do Grid Search
Momentum	0.9	Q2 – melhor do Grid Search
Atributos	Todos (16)	Q5 – melhor F1-Macro
Normalização	Z-score	Pré-processamento padrão
Fração de Treino	100%	Q4 – melhor generalização
Épocas de treinamento	150 (máx.)	Q3–Q7
Validação cruzada	k=10 estratíf.	Q7

Resultado Final com esta configuração: Acurácia = 0.9235 ± 0.0022 | F1-Macro = 0.9210 ± 0.0023 (validação cruzada 10-fold). O modelo é robusto e consistente, sem sinais de overfitting significativo.